Manager de un grupo musical (K-pop)

Equipo		**Grupo**	301
Integrantes	Carlos Uriel Tinajero Sanchez		
Scrum Master Sprint 1		**Fecha de inicio**	14/10/2026

Roles de Scrum

Rol	Quién	Responsabilidades
Product Owner	El docente	Ordena el Product Backlog, aclara dudas de las historias y acepta o devuelve cada historia en la Sprint Review según sus criterios de aceptación.
Scrum Master	Un integrante del equipo (rota en cada sprint)	Facilita los eventos, cuida que el tablero esté actualizado, ayuda a quitar bloqueos y vigila los límites WIP.
Equipo de desarrollo	Los 3 o 4 integrantes del equipo	Estiman, programan, prueban y se revisan entre pares. Todos son responsables de que cada historia quede terminada.

Escala de estimación (puntos de historia)

Los puntos son una medida **relativa** de esfuerzo, no de horas. Usa la serie 1, 2, 3, 5 y compara cada historia con las demás.

- **1 punto:** una función corta con una sola condición o una definición de struct sencilla.
- **2 puntos:** una función con varias líneas, un cambio pequeño en varias partes del programa o una explicación breve.
- **3 puntos:** una función con validaciones y varios casos de prueba.
- **5 puntos:** lógica que combina recorridos, NULL, varias funciones o varios casos límite.

Definición de terminado (DoD)

Una historia pasa a **Hecho** solo si cumple todos sus criterios de aceptación y además:

- Compila con `gcc -Wall -Wextra -o manager manager.c` sin advertencias.
- No usa variables globales.
- Toda función que cambia a una integrante, una canción o un grupo recibe un **apuntador**; ninguna devuelve el struct para “actualizarlo”.
- Las funciones que solo leen datos llevan `const` en el parámetro.
- Todo apuntador que pueda ser NULL se verifica **antes** de usar `->`.
- Se probaron los puntos de la lista “Comprueba que” de la etapa correspondiente.
- Un compañero que no escribió el código lo revisó, y el Product Owner la aceptó en la Sprint Review.

Regla de la práctica: no avances a la siguiente etapa hasta que la anterior pase su lista de comprobación.

Manager de un grupo musical (K-pop)

Un proyecto de desarrollo de software donde aplicarás **Scrum + Kanban** para crear un sistema que ayude a gestionar a un grupo de K-pop.

¿Qué harás?

- Gestionar integrantes, canciones y grupos.
- Implementar estructuras de datos con apuntadores a structs.
- Usar el operador flecha (->).
- Trabajar con structs que apuntan a otros structs y con arreglos.



Metodología ágil

Scrum + Kanban

Scrum
- Planificación
- Daily Scrum
- Revisión
- Retrospectiva

+

Kanban
- Tablero de trabajo
- Límites WIP
- Flujo de tareas
- Visibilidad del progreso



El proyecto se divide en 6 etapas

0. Preparación del equipo
1. Apuntadores a structs — &→
2. Modificar por apuntador
3. Dos apuntadores, NULL e intercambio
4. Structs que apuntan a structs
5. Arreglos de structs y aritmética de apuntadores

Reto final — Integración y presentación

Para lograrlo...

</> Programar → Probar → Revisar en equipo → Presentar

}

Product Backlog

Las historias están ordenadas y respeta las dependencias. El Product Owner puede reordenarlas. La columna **Estado** empieza en "Por hacer": actualízala conforme avance el tablero Kanban.

Prioridad: Alta = imprescindible para el programa · Media = importante para comprender el tema · Baja = refuerzo conceptual. **Pts:** puntos de historia.

ID	Etapa	Historia de usuario	Entregable	Prioridad	Pts	Depende de	Sprint	Estado
HU-00	0	Preparar entorno y tablero	manager.c vacío compilando + tablero Kanban	**Alta**	2	—	Sprint 1	Por hacer
HU-01	1	Definir la estructura Idol	struct Idol + variable jennie	**Alta**	2	HU-00	Sprint 1	Por hacer
HU-02	1	Leer campos con punto, desreferencia y flecha	`Idol *p = &jennie;` + 3 formas de leer fans	**Alta**	3	HU-01	Sprint 1	Por hacer
HU-03	1	Mostrar una integrante	`void mostrar(const Idol *i)`	**Alta**	2	HU-01	Sprint 1	Por hacer
HU-04	2	Limitar un valor a un rango	`int limitar(int valor, int minimo, int maximo)`	**Alta**	1	HU-01	Sprint 1	Por hacer
HU-05	2	Ensayar para ganar popularidad	`void ensayar(Idol *i)`	**Alta**	2	HU-04	Sprint 1	Por hacer
HU-06	2	Descansar para recuperar energía	`void descansar(Idol *i)`	**Alta**	2	HU-04	Sprint 1	Por hacer
HU-07	2	Saber si una integrante está activa	`int esta_activa(const Idol *i)`	**Alta**	1	HU-01	Sprint 1	Por hacer
HU-08	2	Explicar paso por valor y paso por apuntador	Demostración + explicación oral	**Media**	2	HU-05	Sprint 1	Por hacer
HU-09	3	Colaboración segura entre dos integrantes	`int colaboracion(Idol *a, Idol *b)`	**Alta**	3	HU-04	Sprint 2	Por hacer
HU-10	3	Dinámica por turnos con intercambio de apuntadores	Ciclo `while` con `turno` y `otra`	**Media**	5	HU-05, HU-07	Sprint 2	Por hacer
HU-11	3	Explicar intercambio de structs vs. de apuntadores	Diagrama + explicación oral	**Media**	2	HU-10	Sprint 2	Por hacer
HU-12	4	Definir la estructura Cancion	struct Cancion + `discos[2]`	**Alta**	1	HU-01	Sprint 2	Por hacer
HU-13	4	Idol con canción opcional	Campo `Cancion *cancion;` en Idol	**Alta**	2	HU-12	Sprint 2	Por hacer
HU-14	4	Asignar y quitar una canción	`void asignar_cancion(Idol *i, Cancion *c)`	**Alta**	3	HU-13	Sprint 2	Por hacer
HU-15	4	Popularidad total con bono de canción	`int popularidad_total(const Idol *i)`	**Alta**	3	HU-14	Sprint 3	Por hacer
HU-16	4	Definir el Grupo	struct Grupo + grupo "Lumina"	**Alta**	2	HU-13	Sprint 3	Por hacer
HU-17	4	Cambiar la líder del grupo	`int cambiar_lider(Grupo *g, Idol *nueva)`	**Alta**	5	HU-16	Sprint 3	Por hacer

ID	Etapa	Historia de usuario	Entregable	Prioridad	Pts	Depende de	Sprint	Estado
HU-18	5	Mostrar el grupo recorriéndolo con apuntadores	`void mostrar_grupo(const Grupo *g)`	**Media**	3	HU-16	Sprint 3	Por hacer
HU-19	5	Contar integrantes activas	`int contar_activas(const Grupo *g)`	**Media**	2	HU-07, HU-16	Sprint 3	Por hacer
HU-20	5	Encontrar a la integrante más cansada	`Idol *idol_mas_cansada(Grupo *g)`	**Alta**	3	HU-06, HU-16	Sprint 3	Por hacer
HU-21	5	Dar un concierto	`void dar_concierto(Grupo *g)`	**Alta**	5	HU-15, HU-16	Sprint 4	Por hacer
HU-22	5	Explicar la equivalencia entre arreglos y apuntadores	Demostración con `%p` + explicación	**Baja**	2	HU-18	Sprint 4	Por hacer
HU-23	Reto	Mostrar la canción de la líder sin fallar	`void mostrar_cancion_lider(const Grupo *g)`	**Alta**	3	HU-17	Sprint 4	Por hacer
HU-24	Reto	Preparar la presentación del grupo	`void preparar_presentacion(Grupo *g)`	**Alta**	5	HU-18, HU-19, HU-20, HU-21, HU-23	Sprint 4	Por hacer
HU-25	Reto	Justificar el diseño de cada función	Tabla de funciones + compilación final limpia	**Media**	3	HU-24	Sprint 4	Por hacer
			Total del Product Backlog (26 historias)		**69**			

Planeación de sprints

Se proponen cuatro sprints. Los puntos comprometidos por sprint (entre 16 y 18) son una referencia: después del Sprint 1, usa la velocidad real del equipo para decidir cuánto comprometer.

Sprint	Etapas	Objetivo	Historias	Puntos
Sprint 1	Etapas 0, 1 y 2	Tener una `Idol` funcional que se manipula por apuntador: crearla, mostrarla, hacerla ensayar y descansar, y saber si sigue activa.	HU-00 a HU-08 (9 historias)	**17**
Sprint 2	Etapa 3 y primera parte de la Etapa 4	Proteger las funciones contra NULL y apuntadores repetidos, intercambiar apuntadores y empezar a conectar structs creando `Cancion` y asignándola a una `Idol`.	HU-09 a HU-14 (6 historias)	**16**
Sprint 3	Segunda parte de la Etapa 4 y primera parte de la Etapa 5	Completar la relación entre `Idol`, `Cancion` y `Grupo` (popularidad con bono y líder) y recorrer el arreglo de integrantes con aritmética de apuntadores.	HU-15 a HU-20 (6 historias)	**18**
Sprint 4	Final de la Etapa 5 y Reto final	Dar conciertos que gasten energía y canciones, preparar la presentación del grupo y defender las decisiones de diseño del programa.	HU-21 a HU-25 (5 historias)	**18**

Tablero Kanban

Scrum decide **qué** se trabaja en cada sprint, Kanban ayuda a ver **cómo fluye** ese trabajo. Reproduce este tablero en un pizarrón, en papel o en una herramienta digital (por ejemplo Trello, Miro o Jamboard). Cada historia del Sprint Backlog es una tarjeta con su ID, título, puntos, responsable y las fechas de inicio y fin.

Reglas del tablero

- **Límite de trabajo en curso (WIP):** máximo 3 tarjetas en En progreso y 2 en En revisión para todo el equipo. Si una columna está llena, ayuda a terminar antes de empezar algo nuevo.
- **Se jala, no se empuja:** toma una tarjeta nueva solo cuando tienes espacio, siguiendo el orden del Product Backlog.
- **Tarjeta bloqueada:** márcala con una etiqueta roja, avísalo en el Daily y el Scrum Master ayuda a destrabarla.
- **Revisión entre pares:** quien revisa no puede ser quien programó. Si falla un criterio, la tarjeta regresa a En progreso con una nota de qué falta.
- **Mide el flujo:** anota la fecha en que la tarjeta entra a En progreso y la fecha en que llega a Hecho. Ese es su tiempo de ciclo, compáralo entre historias en la retrospectiva.

POR HACER	EN PROGRESO · WIP máx. 3	EN REVISIÓN · WIP máx. 2	HECHO
Entra cuando: Historia con criterios claros, puntos estimados y dentro del sprint.	*Entra cuando: Tiene responsable y cabe en el límite WIP.*	*Entra cuando: Compila sin advertencias y su autor ya probó los criterios.*	*Entra cuando: Un compañero la revisó, cumple la DoD y el Product Owner la aceptó.*
HU-00 · Preparar entorno y tablero (2 pts)			HU-00 Preparar entorno y tablero
HU-01 · Definir la estructura Idol (2 pts)			HU-01 Definir la estructura Idol
HU-02 · Leer campos con punto, desreferencia y flecha (3 pts)			HU-02 · Leer campos con punto, desreferencia y flecha
HU-03 · Mostrar una integrante (2 pts)			HU-03 · Mostrar una integrante
HU-04 · Limitar un valor a un rango (1 pts)			HU-04 · Limitar un valor a un rango
HU-05 · Ensayar para ganar popularidad (2 pts)			HU-05 · Ensayar para ganar popularidad
HU-06 · Descansar para recuperar energía (2 pts)			HU-06 · Descansar para recuperar energía
HU-07 · Saber si una integrante está activa (1 pts)			HU-07 · Saber si una integrante está activa
HU-08 · Explicar paso por valor y paso por apuntador (2 pts)			HU-08 · Explicar paso por valor y paso por apuntador

Tablero inicial con las tarjetas del Sprint 1. Para los siguientes sprints, crea las tarjetas con las historias de la sección 4.

Seguimiento y retrospectiva

Al terminar cada sprint, completa esta tabla. La **velocidad** es el número de puntos de las historias que quedaron en Hecho; úsala para decidir cuánto comprometer en el siguiente sprint.

Sprint	Puntos comprometidos	Puntos terminados (velocidad)	Historias que no terminaron y por qué	Acuerdo de mejora para el siguiente sprint
Sprint 1	17	17	Ninguna. Las 9 historias (HU-00 a HU-08) se completaron. La implementación de limitar facilitó no salirnos de rango con la energía.	Acuerdo: Antes de programar una función que modifica datos, declarar siempre el apuntador y verificar el NULL desde el primer intento para no tener que regresar a corregir.
Sprint 2	16	16	Ninguna. Las 6 historias (HU-09 a HU-14) se completaron. La protección contra NULL en colaboracion evitó que el programa fallara.	Acuerdo: Cuando se agreguen nuevos campos a un struct (como Cancion *cancion), actualizar todos los inicializadores en el main al mismo tiempo para evitar errores de compilación.
Sprint 3	18	18	Ninguna. Las 6 historias (HU-15 a HU-20) se completaron. El uso de aritmética de apuntadores (p++) en mostrar_grupo quedó muy limpio y eficiente.	Acuerdo: Para el siguiente sprint, no dejar la documentación (diagramas o tablas) para el final. Hacer el diagrama de flujo justo después de que la función compile y pase las pruebas.
Sprint 4	18	18	Ninguna. Las 5 historias (HU-21 a HU-25) se completaron. El código final pasó las pruebas de estrés (llamar a preparar_presentacion 3 veces sin que las energías se salieran de rango).	Acuerdo final: Mantener la regla de "revisión entre pares". Aunque el código compiló sin advertencias, una segunda persona siempre encuentra detalles en la lógica de negocio (como el manejo de la vigencia de la canción).

Preguntas guía para la retrospectiva

- ¿Qué hicimos bien que debemos repetir?
- ¿Qué nos frenó o qué tarjetas se quedaron bloqueadas?
- ¿Qué cambio concreto probaremos en el siguiente sprint? (uno solo, que se pueda verificar)

1. ¿Qué hicimos bien que debemos repetir?

Uso estricto de const: Desde el Sprint 1 decidimos que todas las funciones que solo leen datos (como mostrar, esta_activa, popularidad_total) llevarían const en sus parámetros. Esto nos evitó errores accidentales y el compilador nos ayudó a proteger los datos.

Verificación temprana de NULL: En el Sprint 2 y 3 aprendimos a verificar si un apuntador era NULL antes de usar el operador flecha (->). Esto evitó que el programa se cerrara inesperadamente cuando probamos escenarios con grupos vacíos o sin líder.

Aritmética de apuntadores pura: En el Sprint 3 logramos recorrer el arreglo del grupo usando p++ y p < fin, sin usar corchetes [], lo cual demostró que entendimos cómo funciona la memoria y nos ahorró líneas de código.

2. ¿Qué nos frenó o qué tarjetas se quedaron bloqueadas?

Actualización de inicializadores: En el Sprint 2, al agregar el campo Cancion *cancion a la estructura Idol, el programa dejó de compilar porque olvidamos agregar el NULL al final de todas las variables creadas en el main. Tuvimos que regresar a corregir eso.

Confusión de apuntadores vs. structs: En la dinámica por turnos (Sprint 2), al principio intentamos copiar los datos de una integrante a otra, en lugar de solo intercambiar las direcciones de memoria con una variable temporal. Nos frenó tener que repasar el concepto de "paso por referencia".

Diagramas de flujo: La creación de los diagramas a mano nos tomó más tiempo del esperado al final del proyecto, porque los dejamos para el último momento.

3. ¿Qué cambio concreto probaremos en el siguiente sprint? (uno solo, que se pueda verificar)

Acuerdo: "Hacer los diagramas de flujo y la documentación de una función inmediatamente después de que compile y pase sus pruebas, sin dejarla para el final del sprint."

¿Cómo lo verificaremos? En la Sprint Review, el Scrum Master revisará que por cada Historia de Usuario terminada, ya exista su diagrama correspondiente en la wiki o repositorio, antes de mover la tarjeta a la columna de "Hecho".